

Task 4: Mini SQL Agent (Agentic System Task)

Goal

To build a simple **agentic system that thinks, acts, and corrects itself**. A working **AI-powered SQL agent system** that mimics real production-level data assistants like analytics chatbots and BI tools.

GitHub Repo:(Folder: “task4”)

<https://github.com/nilima-sth/text-to-sql-agent-sys.git>

Sheet:

(Sheet Four: Task_4_Gen_sql)

(Sheet Five: Evaluation_for_task4)

[Task_4_Generated_sql](#)

Required Agent Flow

Step 1: Understand Query

- Identify intent
- Detect tables and columns

Step 2: Generate SQL

- Write SQL query based on structure

Step 3: Execute Query

- Run SQL in PostgreSQL

Step 4: Error Handling

If query fails:

- Read error message
- Fix query
- Retry up to **3 times maximum**

Step 5: Final Output

- Return SQL result
- Convert result into natural language summary

Evaluation Criteria

Understanding

- Correct decomposition of queries

Implementation

- Working SQL generation
- Correct API structure

Agent Behavior

- Retry mechanism
- Error handling

System Design

- Clean FastAPI structure
- Logging implementation
- Safe SQL execution

SOME EXAMPLES (SCREENSHOTS)

Example 1: List all the products

Agentic Text-to-SQL Assistant

Ask questions about the provided PostgreSQL schema in natural language.

Enter your query

List all the Products

Run

Answer

Generated SQL

```
SELECT * FROM products;
```

Validated

Yes

Executed

Yes

Workflow details

Plan

Plan:

1. **Source Table:** `products`
2. **Columns:** Select all columns (`*`) or specific identifying columns like `productCode` and `productName` .
3. **Filters:** None.
4. **Output:** A list of all products currently in the database.

SQL

```
SELECT * FROM products;
```

Raw results

Showing only the first 10 rows

```
[
  {
    "productCode": "S10_1678",
    "productName": "1969 Harley Davidson Ultimate Chopper",
    "productLine": "Motorcycles",
    "productScale": "1:16",
    "productVendor": "Min Lin Diecast",
    "productDescription": "Intricately detailed motorcycle model.",
    "quantityInStock": 7933,
    "buyPrice": 48.81,
    "MSRP": 95.7
  },
  {
    "productCode": "S10_1949",
    "productName": "1952 Alpine Renault 1300",
    "productLine": "Classic Cars"
```

Example 2: Get all customers

Agentic Text-to-SQL Assistant

Ask questions about the provided PostgreSQL schema in natural language.

Enter your query

List all customers

Run

Answer

Generated SQL

SELECT * FROM customers;

Validated

Yes

Executed

Yes

Summary

The database contains a comprehensive list of 122 customers from various countries worldwide, including details such as their customer number, name, contact information, address, city, state, postal code, sales representative, and credit limit.

Workflow details

SQL

SELECT * FROM customers;

Raw results

SQL

SELECT * FROM customers;

Raw results

Showing only the first 10 rows

0 : {

"customerNumber" : 103

"customerName" : "Atelier graphique"

"contactLastName" : "Schmitt"

"contactFirstName" : "Carine "

"phone" : "40.32.2555"

"addressLine1" : "54, rue Royale"

"addressLine2" : NULL

"city" : "Nantes"

"state" : NULL

"postalCode" : "44000"

"country" : "France"

"salesRepEmployeeNumber" : 1370

"creditLimit" : 21000

}

1 : {

"customerNumber" : 112

"customerName" : "Signal Gift Stores"

"contactLastName" : "King"

"contactFirstName" : "Jean"

"phone" : "7025551838"

"addressLine1" : "8489 Strong St."

"addressLine2" : NULL

"city" : "Las Vegas"

"state" : "NV"

"postalCode" : "89030"

"country" : "USA"

}

Example 3: Get all offices

Agentic Text-to-SQL Assistant

Ask questions about the provided PostgreSQL schema in natural language.

Enter your query

Get all offices

Run

Answer

Generated SQL

SELECT * FROM offices;

Validated

Yes

Executed

Yes

Summary

There are 7 offices in total, located in San Francisco, Boston, NYC, Paris, Tokyo, Sydney, and London. These offices are distributed across North America, EMEA, Japan, and APAC regions.

Workflow details

SQL

SELECT * FROM offices;

Raw results

Showing only the first 10 rows

0 : {

"officeCode" : "1"

"city" : "San Francisco"

"phone" : "+1 650 219 4782"

"addressLine1" : "100 Market Street"

"addressLine2" : "Suite 300"

"state" : "CA"

"country" : "USA"

"postalCode" : "94080"

"territory" : "NA"

}

1 : {

"officeCode" : "2"

"city" : "Boston"

"phone" : "+1 215 837 0825"

"addressLine1" : "1550 Court Place"

"addressLine2" : "Suite 102"

"state" : "MA"

"country" : "USA"

"postalCode" : "02107"

"territory" : "NA"

}

2 : {

"officeCode" : "3"

"city" : "NYC"

"phone" : "+1 212 555 3000"

"addressLine1" : "523 East 53rd Street"

"addressLine2" : "apt. 5A"

"state" : "NY"

"country" : "USA"

Example 4: Get product names and prices

Agentic Text-to-SQL Assistant

Ask questions about the provided PostgreSQL schema in natural language.

Enter your query

Get product names and prices

Run

Answer

Generated SQL

```
SELECT "productName", "buyPrice" FROM products;
```

Validated

Yes

Executed

Yes

Summary

The database contains a list of 110 products, ranging from vintage vehicles and aircraft to modern motorcycles and ships. The prices vary significantly, with the "1938 Cadillac V-16 Presidential Limousine" being the least expensive at 20.61 and the "1962 LanciaA Delta 16V" being the most expensive at 103.42.

Workflow details

SQL

```
SELECT "productName", "buyPrice" FROM products;
```

Raw results

Showing only the first 10 rows

▼ [📄]

▼ 0 : {

"productName" : "1969 Harley Davidson Ultimate Chopper"

"buyPrice" : 48.81

}

▼ 1 : {

"productName" : "1952 Alpine Renault 1300"

"buyPrice" : 98.58

}

▼ 2 : {

"productName" : "1996 Moto Guzzi 1100i"

"buyPrice" : 68.99

}

▼ 3 : {

"productName" : "2003 Harley-Davidson Eagle Drag Bike"

"buyPrice" : 91.02 📄

}

▼ 4 : {

"productName" : "1972 Alfa Romeo GTA"

"buyPrice" : 85.68

}

▼ 5 : {

"productName" : "1962 LanciaA Delta 16V"

"buyPrice" : 103.42

}

▼ 6 : {

"productName" : "1968 Ford Mustang"

Example 5: Get orders with customer names

Agentic Text-to-SQL Assistant

Ask questions about the provided PostgreSQL schema in natural language.

Enter your query

Get orders with customer names

Run

Answer

Generated SQL

```
SELECT o.*, c."customerName" FROM orders o JOIN customers c ON o."customerNumber" = c."customerNumber";
```

Validated

Yes

Executed

Yes

Summary

The query successfully retrieved a comprehensive list of all orders, including their associated order details (such as order number, dates, and status) and the corresponding customer names. The output contains 326 records, showing a variety of order statuses ranging from "Shipped" and "Resolved" to "Cancelled," "On Hold," and "In Process."

Workflow details

SQL

```
SELECT o.*, c."customerName" FROM orders o JOIN customers c ON o."customerNumber" = c."customerNumber";
```

Raw results

Showing only the first 10 rows

0 : {

"orderNumber" : 10100

"orderDate" : "2003-01-06"

"requiredDate" : "2003-01-13"

"shippedDate" : "2003-01-10"

"status" : "Shipped"

"comments" :

NULL

"customerNumber" : 363

"customerName" : "Online Diecast Creations Co."

}

1 : {

"orderNumber" : 10101

"orderDate" : "2003-01-09"

"requiredDate" : "2003-01-16"

"shippedDate" : "2003-01-11"

"status" : "Shipped"

"comments" : "Check on availability."

"customerNumber" : 128

"customerName" : "Blauer See Auto, Co."

}

2 : {

Example 6: Get payments with customer names

Agentic Text-to-SQL Assistant

Ask questions about the provided PostgreSQL schema in natural language.

Enter your query

Get payments with customer names

Run

Answer

Generated SQL

```
SELECT T1.*, T2."customerName" FROM payments AS T1 JOIN customers AS T2 ON T1."customerNumber" = T2."customerNumber";
```

Validated

Yes

Executed

Yes

Summary

The query successfully retrieved a comprehensive list of all payment records, including the check number, payment date, and amount, joined with the corresponding customer name for each transaction. The results cover a wide range of customers and their respective payment histories.

Workflow details

SQL

```
SELECT T1.*, T2."customerName" FROM payments AS T1 JOIN customers AS T2 ON T1."customerNumber" = T2."customerNumber";
```

Raw results

Showing only the first 10 rows

0 : {

"customerNumber" : 103

"checkNumber" : "HQ336336"

"paymentDate" : "2004-10-19"

"amount" : 6066.78

"customerName" : "Atelier graphique"

}

1 : {

"customerNumber" : 103

"checkNumber" : "JM553205"

"paymentDate" : "2003-06-05"

"amount" : 14571.44

"customerName" : "Atelier graphique"

}

2 : {

"customerNumber" : 103

"checkNumber" : "OM314933"

"paymentDate" : "2004-12-18"

"amount" : 1676.14

"customerName" : "Atelier graphique"

}

3 : {

"customerNumber" : 112

Example 7: Get products with product line description

Agentic Text-to-SQL Assistant

Ask questions about the provided PostgreSQL schema in natural language.

Enter your query

Get products with product line description

Run

Answer

Generated SQL

```
SELECT p.*, pl."textDescription" FROM products p JOIN productlines pl ON p."productLine" = pl."productLine";
```

Validated

Yes

Executed

Yes

Summary

The query successfully retrieved a comprehensive list of all products, including their specific details (such as product name, vendor, stock quantity, and pricing) alongside the descriptive text associated with their respective product lines (e.g., "Motorcycles," "Classic Cars," "Vintage Cars," "Trucks and Buses," "Planes," "Ships," and "Trains").

Workflow details

SQL

```
SELECT p.*, pl."textDescription" FROM products p JOIN productlines pl ON p."productLine" = pl."productLine";
```

Raw results

Showing only the first 10 rows

0 : {

"productCode" : "S10_1678"

"productName" : "1969 Harley Davidson Ultimate Chopper"

"productLine" : "Motorcycles"

"productScale" : "1:10"

"productVendor" : "Min Lin Diecast"

"productDescription" :
"This replica features working kickstand, front suspension, gear-shift lever, footbrake lever, drive chain, wheels and steering. All parts are particularly delicate due to their precise scale and require special care and attention."
⌵

"quantityInStock" : 7933

"buyPrice" : 48.61

"MSRP" : 95.7

"textDescription" :
"Our motorcycles are state of the art replicas of classic as well as contemporary motorcycle legends such as Harley Davidson, Ducati and Vespa. Models contain stunning details such as official logos, rotating wheels, working kickstand, front suspension, gear-shift lever, footbrake lever, and drive chain. Materials used include diecast and plastic. The models range in size from 1:10 to 1:50 scale and include numerous limited edition and several out-of-production vehicles. All models come fully assembled and ready for display in the home or office. Most include a certificate of authenticity."
⌵

}

1 : {

"productCode" : "S10_1949"

"productName" : "1952 Alpine Renault 1300"

"productLine" : "Classic Cars"

"productScale" : "1:10"

Example 8: Get payments with customer country

Agentic Text-to-SQL Assistant

Ask questions about the provided PostgreSQL schema in natural language.

Enter your query

Get payments with customer country

Run

Answer

Generated SQL

```
SELECT p.*, c.country FROM payments p JOIN customers c ON p."customerNumber" = c."customerNumber";
```

Validated

Yes

Executed

Yes

Summary

The query successfully retrieved a comprehensive list of all payment records, including the check number, payment date, and amount, joined with the corresponding country of the customer who made each payment. The results cover a diverse range of international customers from countries such as France, USA, Australia, Norway, Germany, Spain, Sweden, Denmark, Singapore, Japan, Finland, UK, Ireland, Canada, Hong Kong, Italy, Switzerland, Belgium, Austria, New Zealand, and the Philippines.

Workflow details

SQL

```
SELECT p.*, c.country FROM payments p JOIN customers c ON p."customerNumber" = c."customerNumber";
```

Raw results

Showing only the first 10 rows

```
[{"customerNumber": 103, "checkNumber": "HQ336336", "paymentDate": "2004-10-19", "amount": 6966.78, "country": "France"}, {"customerNumber": 103, "checkNumber": "JM555285", "paymentDate": "2003-06-05", "amount": 14571.44, "country": "France"}, {"customerNumber": 103, "checkNumber": "OM314933", "paymentDate": "2004-12-18", "amount": 1676.14, "country": "France"}]
```

Example 9: Number of orders per status

Agentic Text-to-SQL Assistant

Ask questions about the provided PostgreSQL schema in natural language.

Enter your query

Number of orders per status

Run

Answer

Generated SQL

SELECT status, COUNT(*) FROM orders GROUP BY status;

Validated

Yes

Executed

Yes

Summary

The database contains a total of 326 orders categorized by status. The vast majority of orders are "Shipped" (303), followed by a small number of orders that are "In Process" (6), "Cancelled" (6), "Resolved" (4), "On Hold" (4), and "Disputed" (3).

Workflow details

SQL

SELECT status, COUNT(*) FROM orders GROUP BY status;

Raw results

Showing only the first 10 rows

0 : {

"status" : "Shipped"

"count" : 303

}

1 : {

"status" : "In Process"

"count" : 6

}

2 : {

"status" : "Disputed"

"count" : 3

}

3 : {

"status" : "Cancelled"

"count" : 6

}

4 : {

"status" : "Resolved"

"count" : 4

}

5 : {

"status" : "On Hold"

"count" : 4

}

Example 10: Products per product line

Agentic Text-to-SQL Assistant

Ask questions about the provided PostgreSQL schema in natural language.

Enter your query

Products per product line

Run

Answer

Generated SQL

```
SELECT "productLine", COUNT(*) FROM products GROUP BY "productLine";
```

Validated

Yes

Executed

Yes

The database contains 110 products distributed across seven categories. Classic Cars is the largest category with 38 products, followed by Vintage Cars (24), Motorcycles (13), Planes (12), Trucks and Buses (11), Ships (9), and Trains (3).

Workflow details

SQL

```
SELECT "productLine", COUNT(*) FROM products GROUP BY "productLine";
```

Raw results

Showing only the first 10 rows

0 : {

"productLine" : "Classic Cars"

"count" : 38

}

1 : {

"productLine" : "Trains"

"count" : 3

}

2 : {

"productLine" : "Planes"

"count" : 12

}

Example 11: Employees per office

Agentic Text-to-SQL Assistant

Ask questions about the provided PostgreSQL schema in natural language.

Enter your query

Employees per office

Run

Answer

Generated SQL

```
SELECT "officeCode", COUNT("employeeNumber") AS employee_count FROM employees GROUP BY "officeCode";
```

Validated

Yes

Executed

Yes

Summary

The employee distribution across offices is as follows: office 1 has 6 employees, office 4 has 5 employees, office 6 has 4 employees, while offices 2, 3, 5, and 7 each have 2 employees.

Workflow details

SQL

```
SELECT "officeCode", COUNT("employeeNumber") AS employee_count FROM employees GROUP BY "officeCode";
```

Raw results

Showing only the first 10 rows

```
[
  {
    "officeCode": "6",
    "employee_count": 4
  },
  {
    "officeCode": "2",
    "employee_count": 2
  },
  {
    "officeCode": "4",
    "employee_count": 5
  },
  {
    "officeCode": "7",
    "employee_count": 2
  },
  {
    "officeCode": "5",
    "employee_count": 2
  }
]
```

Example 12: Total stock per product vendor

Agentic Text-to-SQL Assistant

Ask questions about the provided PostgreSQL schema in natural language.

Enter your query

Total stock per product vendor

Run

Answer

Generated SQL

```
SELECT "productVendor", SUM("quantityInStock") AS "totalStock" FROM products GROUP BY "productVendor";
```

Validated

Yes

Executed

Yes

Summary

The database contains stock data for 13 different product vendors. Gearbox Collectibles holds the highest inventory with 60,495 units, followed by Min Lin Diecast with 50,089 units. Autoart Studio Design has the lowest inventory among the listed vendors, totaling 30,093 units.

Workflow details

SQL

```
SELECT "productVendor", SUM("quantityInStock") AS "totalStock" FROM products GROUP BY "productVendor";
```

Raw results

Showing only the first 10 rows

```
[{"productVendor": "Welly Diecast Productions", "totalStock": 45895}, {"productVendor": "Motor City Art Classics", "totalStock": 43185}, {"productVendor": "Classic Metal Creations", "totalStock": 45488}
```

Example 13: Max MSRP per product line

Agentic Text-to-SQL Assistant

Ask questions about the provided PostgreSQL schema in natural language.

Enter your query

Max MSRP per product line

Run

Answer

Generated SQL

```
SELECT "productLine", MAX("MSRP") FROM products GROUP BY "productLine";
```

Validated

Yes

Executed

Yes

Summary

The maximum MSRP values across product lines are as follows: Classic Cars at 214.3, Motorcycles at 193.66, Vintage Cars at 170.0, Planes at 157.69, Trucks and Buses at 136.67, Ships at 122.89, and Trains at 100.84.

Workflow details

SQL

```
SELECT "productLine", MAX("MSRP") FROM products GROUP BY "productLine";
```

Raw results

Showing only the first 10 rows

0 : {

"productLine" : "Classic Cars"

"max" : 214.3

}

1 : {

"productLine" : "Trains"

"max" : 100.84

}

2 : {

"productLine" : "Planes"

"max" : 157.69

}

3 : {

"productLine" : "Trucks and Buses"

"max" : 136.67

}

4 : {

"productLine" : "Vintage Cars"

"max" : 170

}

Example 14: Total revenue from payments

Agentic Text-to-SQL Assistant

Ask questions about the provided PostgreSQL schema in natural language.

Enter your query

Total revenue from payments

Run

Answer

Generated SQL

```
SELECT SUM(amount) FROM payments;
```

Validated

Yes

Executed

Yes

Summary

The total revenue generated from all payments is 8,853,839.23.

Workflow details

SQL

```
SELECT SUM(amount) FROM payments;
```

Raw results

Showing only the first 10 rows

```
[{"@": {"sum": 8853839.23}]
```


Example 15: Number of employees

Agentic Text-to-SQL Assistant

Ask questions about the provided PostgreSQL schema in natural language.

Enter your query

Number of employees

Run

Answer

Generated SQL

```
SELECT COUNT(*) FROM employees;
```

Validated

Yes

Executed

Yes

Summary

There are currently 23 employees recorded in the database.

Workflow details

SQL

```
SELECT COUNT(*) FROM employees;
```

Raw results

Showing only the first 10 rows

```
[
  [
    {
      "count": 23
    }
  ]
]
```

EVALUATION MATRIX

Qsn	sql	gen_sql	Valid ated	Criteria: Understandin g	Criteria: Implementatio n	Compariso n_Notes	Criteria: Agent Behavior	Criteria: System Design
List all prod ucts	SELE CT * FROM produ cts	SELECT * FROM products;	Yes	Yes	Yes	Passed (Correctly decompose d logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.
Get all custo mers	SELE CT * FROM custo mers	SELECT * FROM customers;	Yes	Yes	Yes	Passed (Correctly decompose d logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.
Sho w all order s	SELE CT * FROM orders	SELECT * FROM orders;	Yes	Yes	Yes	Passed (Correctly decompose d logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.
List all empl oyee s	SELE CT * FROM emplo yees	SELECT * FROM employees;	Yes	Yes	Yes	Passed (Correctly decompose d logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.
Get all office s	SELE CT * FROM offices	SELECT * FROM offices;	Yes	Yes	Yes	Passed (Correctly decompose d logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.
Sho w all prod uct lines	SELE CT * FROM produ ctlines	SELECT * FROM productlines ;	Yes	Yes	Yes	Passed (Correctly decompose d logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.
List all pay ment s	SELE CT * FROM paym ents	SELECT * FROM payments;	Yes	Yes	Yes	Passed (Correctly decompose d logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.

Get product names and prices	SELECT productName, buyPrice FROM products;	SELECT "productName", "buyPrice" FROM products;	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) FastAPI/Logging require code review
Get customer names and cities	SELECT customerName, city FROM customers;	SELECT "customerName", "city" FROM customers;	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) FastAPI/Logging require code review
List employee first and last names	SELECT firstName, lastName FROM employees;	SELECT "firstName", "lastName" FROM employees;	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) FastAPI/Logging require code review
Get all order dates	SELECT orderDate FROM orders	SELECT "orderDate" FROM orders;	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)
Show product vendor list	SELECT DISTINCT productVendor FROM products	SELECT DISTINCT "productVendor" FROM products;	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)
Get all product	SELECT productCode FROM products;	SELECT "productCode" FROM products;	Yes	Yes	Passed (Correctly decomposed logic into valid	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling

code s	e FROM produ cts				tables & columns)			leading to 100% Validated status)
List all coun tries from office s	SELE CT DISTI NCT countr y FROM offices	SELECT DISTINCT country FROM offices;	Yes	Yes	Yes	Passed (Correctly decompose d logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.
Sho w all order statu ses	SELE CT DISTI NCT status FROM orders	SELECT DISTINCT status FROM orders;	Yes	Yes	Yes	Passed (Correctly decompose d logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.
Get all pay ment amo unts	SELE CT amou nt FROM paym ents	SELECT amount FROM payments;	Yes	Yes	Yes	Passed (Correctly decompose d logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.
List all job titles	SELE CT DISTI NCT jobTitl e FROM emplo yees	SELECT DISTINCT "jobTitle" FROM employees;	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)
Get custo mer phon e num bers	SELE CT phone FROM custo mers	SELECT "phone" FROM customers;	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)
Sho w prod uct MSRP	SELE CT produ ctNam e, MSRP	SELECT "MSRP" FROM products;	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Properly quotes identifiers for safe parsing.	Passed (Assumed robust error handling leading to	Safe SQL (SELECT only) FastAPI/Loggi ng require code review

values	FROM products;						100% Validated status)	
List order numbers	SELECT order Number FROM orders	SELECT "orderNumber" FROM orders;	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)
Get orders with customer names	SELECT o.orderNumber, c.customerName FROM orders o JOIN customers c ON o.customerNumber = c.customerNumber;	SELECT o.*, c."customerName" FROM orders o JOIN customers c ON o."customerNumber" = c."customerNumber";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Selects all columns (`*`) instead of strictly matching benchmark columns. Properly quotes identifiers for safe parsing.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) FastAPI/Logging require code review
Get employees with office city	SELECT e.firstName, e.lastName, o.city FROM employees e JOIN offices o ON e.officeCode = o.officeCode;	SELECT e."firstName", e."lastName", o."city" FROM employees e JOIN offices o ON e."officeCode" = o."officeCode";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Properly quotes identifiers for safe parsing.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) FastAPI/Logging require code review

Get payments with customer names	SELECT p.amount, c.customerName FROM payments p JOIN customers c ON p.customerNumber = c.customerNumber;	SELECT T1.*, T2."customerName" FROM payments AS T1 JOIN customers AS T2 ON T1."customerNumber" = T2."customerNumber";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Selects all columns (`*`) instead of strictly matching benchmark columns. Properly quotes identifiers for safe parsing. Utilizes standardized T1/T2 aliasing. Adds explicit aliases (e.g. AS total_orders).	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) FastAPI/Logging require code review
Get order details with product names	SELECT od.orderNumber, p.productName FROM orderdetails od JOIN products p ON od.productCode = p.productCode;	SELECT T1.*, T2."productName" FROM orderdetails T1 JOIN products T2 ON T1."productCode" = T2."productCode";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Selects all columns (`*`) instead of strictly matching benchmark columns. Properly quotes identifiers for safe parsing. Utilizes standardized T1/T2 aliasing.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) FastAPI/Logging require code review
Get products with product line desc	SELECT p.*, pl."textDescription" FROM products p JOIN productlines	SELECT p.*, pl."textDescription" FROM products p JOIN productlines	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Selects all columns (`*`) instead of strictly matching benchmark columns. Properly	Passed (Assumed robust error handling leading to 100%	Safe SQL (SELECT only) FastAPI/Logging require code review

ription	FROM products p JOIN productlines pl ON p.productLine=pl.productLine;	pl ON p."productLine" = pl."productLine";				quotes identifiers for safe parsing.	Validated status)	
Get customers with sales rep names	SELECT c.customerName, e.firstName, e.lastName FROM customers c JOIN employees e ON c.salesRepEmployeeNumber=e.employeeNumber;	SELECT c."customerName", e."firstName", e."lastName" FROM customers c JOIN employees e ON c."salesRepEmployeeNumber" = e."employeeNumber";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Properly quotes identifiers for safe parsing.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) FastAPI/Logging require code review
Get orders with customer city	SELECT o.orderNumber, c.city FROM orders o JOIN customers c ON o.customerNumber=	SELECT T1."orderNumber", T2."city" FROM orders AS T1 JOIN customers AS T2 ON T1."customerNumber" = T2."customerNumber";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Properly quotes identifiers for safe parsing. Utilizes standardized T1/T2 aliasing. Adds explicit aliases (e.g. AS total_orders)	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) FastAPI/Logging require code review

	omerN umber =c.cus tomer Numb er;							
Get empl oyee s and their man ager	SELE CT e.first Name, m.first Name AS mana ger FROM emplo yees e LEFT JOIN emplo yees m ON e.repo rtsTo= m.em ployee Numb er;	SELECT e."firstName " AS employee_fi rst_name, e."lastName " AS employee_l ast_name, m."firstNam e" AS manager_fir st_name, m."lastNam e" AS manager_la st_name FROM employees e LEFT JOIN employees m ON e."reportsTo " = m."employee eNumber";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Properly quotes identifiers for safe parsing.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) FastAPI/Loggi ng require code review
Get order detail s with prod uct vend or	SELE CT od.ord erNum ber, p.prod uctVe ndor FROM orderd etails od JOIN produ cts p ON od.pro ductC	SELECT T1.*, T2."product Vendor" FROM orderdetails T1 JOIN products T2 ON T1."product Code" = T2."product Code";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Selects all columns (``) instead of strictly matching benchmark columns. Properly quotes identifiers for safe parsing. Utilizes standardize d T1/T2 aliasing.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) FastAPI/Loggi ng require code review

	ode=p .produ ctCod e;							
Get pay ment s with custo mer coun try	SELE CT p.amo unt, c.coun try FROM paym ents p JOIN custo mers c ON p.cust omerN umber =c.cus tomer Numb er;	SELECT p.*, c.country FROM payments p JOIN customers c ON p."customer Number" = c."customer Number";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Selects all columns (`) instead of strictly matching benchmark columns. Properly quotes identifiers for safe parsing.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) FastAPI/Loggi ng require code review
Coun t custo mers per coun try	SELE CT countr y, COUN T(*) FROM custo mers GROU P BY countr y;	SELECT country, COUNT(*) FROM customers GROUP BY country;	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) FastAPI/Loggi ng require code review
Total pay ment s per custo mer	SELE CT custo merNu mber, SUM(amou nt) FROM paym ents GROU P BY custo	SELECT "customerN umber", SUM(amou nt) AS total_payme nts FROM payments GROUP BY "customerN umber";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Properly quotes identifiers for safe parsing. Adds explicit aliases (e.g. AS total_orders .	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) FastAPI/Loggi ng require code review

	merNumber;							
Number of orders per status	SELECT status, COUNT(*) FROM orders GROUP BY status;	SELECT status, COUNT(*) FROM orders GROUP BY status;	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) FastAPI/Logging require code review
Products per product line	SELECT productLine, COUNT(*) FROM products GROUP BY productLine;	SELECT "productLine", COUNT(*) FROM products GROUP BY "productLine";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) FastAPI/Logging require code review
Employees per office	SELECT officeCode, COUNT(*) FROM employees GROUP BY officeCode;	SELECT "officeCode", COUNT("employeeNumber") AS employee_count FROM employees GROUP BY "officeCode";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Properly quotes identifiers for safe parsing. Adds explicit aliases (e.g. AS total_orders).	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) FastAPI/Logging require code review
Total stock per product vendor	SELECT productVendor, SUM(quantityInStock) AS "totalStock" FROM products GROUP BY productVendor;	SELECT "productVendor", SUM("quantityInStock") AS "totalStock" FROM products GROUP BY "productVendor";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Properly quotes identifiers for safe parsing. Adds explicit aliases (e.g. AS total_orders).	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) FastAPI/Logging require code review

	GROUP BY productVendor;							
Average buy price per product line	SELECT productLine, AVG(buyPrice) FROM products GROUP BY productLine;	SELECT "productLine", AVG("buyPrice") FROM products GROUP BY "productLine";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) FastAPI/Logging require code review
Orders per customer	SELECT customerNumber, COUNT(*) FROM orders GROUP BY customerNumber;	SELECT "customerNumber", COUNT("orderNumber") AS total_orders FROM orders GROUP BY "customerNumber";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Properly quotes identifiers for safe parsing. Adds explicit aliases (e.g. AS total_orders).	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) FastAPI/Logging require code review
Max MSRP per product line	SELECT productLine, MAX(MSRP) FROM products GROUP BY productLine;	SELECT "productLine", MAX("MSRP") FROM products GROUP BY "productLine";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) FastAPI/Logging require code review

Min buy price per vendor	SELECT productVendor, MIN(buyPrice) FROM products GROUP BY productVendor;	SELECT "productVendor", MIN("buyPrice") FROM products GROUP BY "productVendor";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) FastAPI/Logging require code review
Total number of customers	SELECT COUNT(*) FROM customers	SELECT COUNT(*) FROM customers;	Yes	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.
Total number of products	SELECT COUNT(*) FROM products	SELECT COUNT(*) FROM products;	Yes	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.
Total revenue from payments	SELECT SUM(amount) FROM payments	SELECT SUM(amount) FROM payments;	Yes	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.
Average product price	SELECT AVG(buyPrice) FROM products	SELECT AVG("buyPrice") FROM products;	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)
Max payment	SELECT MAX(amount) FROM payments	SELECT MAX(amount) FROM payments;	Yes	Yes	Yes	Passed (Correctly decomposed logic into	Passed (Valid SQL executed)	Exact match.

amount	nt) FROM payments					valid tables & columns)		
Min pay ment amount	SELE CT MIN(a mount) FROM paym ents	SELECT MIN(amoun t) FROM payments;	Yes	Yes	Yes	Passed (Correctly decompose d logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.
Coun t total order s	SELE CT COUN T(*) FROM orders	SELECT COUNT(*) FROM orders;	Yes	Yes	Yes	Passed (Correctly decompose d logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.
Total quan tity in stock	SELE CT SUM(quantityInSt ock) FROM produ cts	SELECT SUM("quant ityInStock") FROM products;	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)
Aver age MSRP	SELE CT AVG(MSRP) FROM produ cts	SELECT AVG("MSRP") FROM products;	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)
Num ber of empl oyees	SELE CT COUN T(*) FROM emplo yees	SELECT COUNT(*) FROM employees;	Yes	Yes	Yes	Passed (Correctly decompose d logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.